Trabajo Práctico Integrador

Alumnos

Cristian A. Morales, Javier Ovelar, Bruno Mazza, Gastón Paschetta

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Metodología de Sistemas 2

Docente Titular

Aristiaran Martín León

Docente Tutor

Lobos Diego

8 de noviembre de 2026

Índice

Repositorio de Github
Video
1. Resumen ejecutivo
2. Descripción del sistema base
3. Calidad externa e interna
    Calidad externa
    Calidad interna
4. Diagnóstico de deuda técnica y code smells
    Hallazgos principales
5. Aplicación de SOLID y Clean Code
6. Refactorings aplicados
    Introduce Parameter Object (Gaston)
    Move Method (Javier)
    Extract Method (Bruno)
    Conexión inyectada (Cristian, transversal)
7. Pruebas y verificación
    Cómo se armó la suite
    Otras verificaciones
8. Code review y checklist técnico
    Checklist
    Observaciones y correcciones
9. Uso de inteligencia artificial
10. Conclusiones y mejoras futuras
    Conclusiones grupales
    Mejoras pendientes
    Conclusiones individuales
11. Anexos y enlaces
Referencias

Repositorio de Github

Link

Video

Link

1. Resumen ejecutivo

Partimos de una API de Gestión de Solicitudes Académicas (FastAPI + SQLite) escrita en un único `main.py` de 361 líneas, con conexión global, SQL mezclado con la lógica y errores devueltos con HTTP 200. Repartimos el código en cuatro partes y cada integrante pasó por todas las etapas sobre la suya. El diagnóstico ejecutó 78 casos y registró 47 hallazgos, 16 de prioridad alta, entre ellos una inyección SQL y varios errores 500. Antes de tocar el código fijamos el comportamiento con 64 tests de línea de base. Aplicamos 23 refactorings documentados —entre ellos Extract Method, Move Method e Introduce Parameter Object— que llevaron el sistema a una estructura por módulos con conexión inyectada. La suite final de 74 tests pasa sin modificar ningún assert, y los defectos que cambian la salida quedaron registrados como mejoras pendientes.

2. Descripción del sistema base

El sistema registra las solicitudes que los estudiantes hacen a la facultad (certificados, equivalencias, mesas especiales, reclamos y bajas), las deriva al área que corresponde, permite asignar un responsable y seguir su estado hasta que se resuelve.

- **Actores:** estudiantes (inician las solicitudes), responsables de cada área (las atienden) y personal administrativo (asigna, consulta y saca el reporte).
- **Entidades:** `estudiantes`, `responsables`, `solicitudes` e `historial` (cada cambio de estado).
- **Valores del dominio:** 5 tipos (`CERT`, `EQUIV`, `MESA`, `RECLAMO`, `BAJA`), 4 áreas (Alumnado, Departamento Académico, Bedelia, Coordinación) y 6 estados (`PEND`, `ASIG`, `EN_PROC`, `RES`, `RECH`, `CANC`).
- **Tecnología:** Python, FastAPI, SQLite y pytest. Datos de prueba sintéticos.
- **Versión inicial:** tag `estado-inicial`, un único `main.py` de 361 líneas.

Operación	Endpoint	Responsable en el TP
Alta y listado de estudiantes	`POST/GET /estudiantes`	Cristian
Alta y listado de responsables	`POST/GET /responsables`	Cristian
Registrar solicitud (prioridad y área automáticas)	`POST /solicitudes`	Gaston
Cambiar estado	`PUT /solicitudes/{id}/estado`	Javier
Asignar responsable	`PUT /solicitudes/{id}/responsable`	Javier
Buscar y filtrar	`GET /solicitudes`	Bruno
Detalle con historial	`GET /solicitudes/{id}`	Bruno
Reporte (totales y vencidas)	`GET /reporte`	Bruno

Alcance del trabajo: no se agregó funcionalidad. Se diagnosticó la calidad, se fijó el comportamiento con tests y se refactorizó conservando la misma salida. El reparto fue por código y no por etapa: cada integrante hizo el diagnóstico, los tests y el refactoring de su parte, y además coordinó una tarea común (Gaston, la tabla de smells; Bruno, los tests; Javier, el code review y `docs/`; Cristian, los cambios transversales, el informe, los tags y el video).

Evidencia de funcionamiento: capturas de Swagger UI de cada caso en `docs/evidencias/fase1/` del repositorio.

3. Calidad externa e interna

Desde afuera el sistema funciona: los ocho casos de uso responden y los flujos normales dan el resultado esperado. Los problemas aparecen con datos fuera de lo común y, sobre todo, en la estructura interna, que hace caro y riesgoso cualquier cambio.

Calidad externa

Medida con 78 casos ejecutados desde Swagger UI, con el criterio común de códigos HTTP del grupo.

- Los flujos normales funcionan: altas, búsqueda, detalle, cambios de estado válidos y reporte.
- Los errores responden con **HTTP 200** y un `{"error": ...}`, así que un cliente no puede distinguir éxito de fallo por el código.
- Hay **errores 500** con datos válidos para JSON: `email: null`, `descripcion` nula o numérica, fechas que el reporte no interpreta.
- Se aceptan datos inválidos: email `"@"`, apellido nulo, legajos nulos repetidos, responsables duplicados.
- La búsqueda es vulnerable a **inyección SQL**.

Calidad interna

Lo que hay adentro	Por qué importa para mantenerlo y hacerlo evolucionar
Conexión y cursor globales; importar `main` crea la base y carga datos	No se puede usar otra base sin trucos; los tests dependían de un efecto secundario
Todo en un archivo de 361 líneas	Cuatro personas cambiando el mismo archivo: conflictos en cada merge
SQL, validación, reglas de negocio y respuesta en la misma función	Cada cambio toca todo; no se puede probar una regla por separado
Columnas leídas por posición (`r[3]`, `s[5]`)	Reordenar una columna rompe respuestas sin dar error
Código duplicado: validación de email, conversión de filas, historial y notificación	Un arreglo en un lugar deja el otro con el defecto
Valores del dominio repetidos como strings (áreas, estados, transiciones)	Agregar un área o un estado obliga a buscar todos los lugares
`guardar_solicitud` con 11 parámetros posicionales	Invertir dos del mismo tipo no da error
`except: pass` en la notificación	Los fallos desaparecen sin rastro

La relación entre las dos miradas es directa: varios defectos visibles (los 200, los 500, los datos inválidos aceptados) salen de problemas internos como la falta de un modelo de entrada y la validación repetida a mano en cada endpoint.

4. Diagnóstico de deuda técnica y code smells

Registramos 47 hallazgos, 16 de prioridad alta. Cada integrante ejecutó casos sobre su parte, capturó la evidencia y completó una ficha con los seis campos de la consigna, más el principio que se viola y el refactoring propuesto.

Criterios comunes: líneas siempre referidas al tag `estado-inicial`; prioridad **Alta** = defecto comprobado en los casos o problema que impide escribir tests confiables; **Media** = dificulta el cambio sin falla comprobada; **Baja** = legibilidad.

Integrante	Parte	Casos	Hallazgos	Alta
Cristian	conexión, tablas, estudiantes, responsables (líneas 1–111)	15	13	5
Gaston	registrar solicitud (114–198)	17	9	2
Javier	cambio de estado y asignación (251–318)	26	12	5
Bruno	búsqueda, detalle y reporte (201–248, 321–361)	20	13	4
Total		**78**	**47**	**16**

Hallazgos principales

Uno o más por parte; el detalle completo de los 47 está en `docs/diagnostico/<nombre>_ficha_smells.md`.

#	Smell	Evidencia y ubicación	Impacto y riesgo	Prioridad
C1	Global State	`conn = sqlite3.connect(...)` a nivel módulo (líneas 7–8)	Todo el sistema comparte una conexión; no se puede cambiar la base ni aislar tests	Alta
C2	Efectos secundarios al importar	`import main` crea tablas y carga datos (líneas 10–54)	Los tests dependían de cambiar de carpeta y reimportar el módulo	Alta
C3, G4, J1	Manejo de errores inconsistente	`return {"error": "falta apellido"}` con HTTP 200 (22 lugares en tres partes)	El cliente no distingue éxito de error; el criterio cambia según el endpoint	Alta
C5, G5, J5	Falta de modelo de entrada	`data: dict = Body(...)` y validación a mano en todos los `POST` y `PUT`	Datos inválidos aceptados y errores 500 (`email: null`, `descripcion` numérica)	Alta
B1	Inyección SQL	`q += " AND estado = '" + estado + "'"` (líneas 205–222)	Un filtro puede evadirse con texto armado; riesgo de seguridad	Alta
J3, J6	Lógica de estados repartida; reasignación sin historial	`asignar` cambia `PEND → ASIG` por su cuenta (líneas 309–316)	`PEND → ASIG` por `/estado` nunca puede tener éxito; se pierde trazabilidad	Alta
G1	Long Parameter List	`guardar_solicitud(...)` con 11 posicionales (líneas 116–119)	Invertir dos argumentos del mismo tipo no da error	Media

#	Smell	Evidencia y ubicación	Impacto y riesgo	Prioridad
G6, J2	Switch Statements	cadenas de `if` por tipo y por estado (líneas 150–174 y 259–266)	Agregar un tipo o un estado obliga a editar varios `if`	Media
C9, B3	Duplicated Code: conversión de filas	el mismo diccionario de 12 campos escrito dos veces	Un cambio en la tabla se corrige en un lugar y se olvida en el otro	Media / Alta
C8, B4, J9	Magic Numbers: índices de columnas	`r[3]`, `s[5]`, `est[4]` en las cuatro partes	Reordenar columnas rompe respuestas sin error	Media / Alta
B5, G3, J11	Long Method / SRP	`reporte`, `registrar` y `proc_estado` hacen todo en una función	Imposible probar o cambiar una regla por separado	Media
G7, J7	Excepción silenciada	`except: pass` al notificar (líneas 190–194 y 282–286)	Los fallos de notificación desaparecen sin rastro	Media

Las fichas, las tablas de resultados con cada caso y las capturas están en `docs/diagnostico/` y `docs/evidencias/fase1/`. Tag del cierre de la etapa: `etapa-diagnostico`.

5. Aplicación de SOLID y Clean Code

La mejora principal fue separar responsabilidades: de un archivo con todo mezclado a módulos con un propósito cada uno, con la conexión inyectada y los valores del dominio en un solo lugar.

```
main.py                 crea la aplicación (crear_app) y registra los routers
app/
  db.py                 conexión, tablas y dependencia get_db
  seed.py               datos de prueba
  dominio.py            AREAS, REGLAS_POR_TIPO, TRANSICIONES, ESTADOS_CERRADOS
  validaciones.py       validaciones compartidas
  routers/              estudiantes, responsables, solicitudes, estados,
consultas
```

Principio	Dónde se aplicó	Ejemplo
SRP	Un router por área; funciones que hacían todo, divididas	`crear_estudiante` delega en `validar_estudiante`, `legajo_existe` e `insertar_estudiante`; `reporte` en `contar`, `dias_limite` y `porcentaje_resueltas`; la notificación y el historial salen de `registrar` y `cambiar_estado`
OCP	Reglas del dominio como datos	Agregar un tipo de solicitud, un área o una transición es sumar una entrada en `REGLAS_POR_TIPO`, `AREAS` o `TRANSICIONES`, sin tocar los endpoints
DIP	La conexión se inyecta	Los endpoints reciben `conn` con `Depends(get_db)`; `crear_app(ruta_db)` decide la base, así los tests usan una temporal sin trucos
LSP / ISP	No corresponden	El sistema no tiene jerarquías de clases ni interfaces que sustituir o segregar; forzarlas habría agregado complejidad sin resolver ningún hallazgo

Clean Code	Qué se hizo
Nombres	`nuevoResponsable` → `crear_responsable`, `get_resp` → `listar_responsables`, `proc_estado` → `cambiar_estado`, `asignar` → `asignar_responsable`; variables `p`, `a`, `s`, `f` con nombres que dicen qué son
Funciones	Funciones cortas, con un solo propósito; `guardar_solicitud` pasó de 11 parámetros a un objeto `NuevaSolicitud`
Manejo de errores	Se aplicó lo que no cambia la respuesta: validación separada del resto y notificación aislada. Cambiar los códigos HTTP quedó como mejora pendiente (sección 10)
Cohesión y acoplamiento	Columnas leídas por nombre (`fila["legajo"]`) en lugar de por posición; validación de email en un solo lugar; código muerto y comentado eliminado
Límites entre componentes	Acceso a datos (`db.py`), reglas (`dominio.py`), validación (`validaciones.py`) y HTTP (`routers/`) en módulos distintos. Las decisiones están en `docs/decisiones/01_estructura_y_conexion.md`

6. Refactorings aplicados

Aplicamos 23 refactorings documentados, cada uno en su propio commit y con los tests en verde después de cada uno. Los tres obligatorios están cubiertos: **Extract Method** (Bruno), **Move Method** (Javier) e **Introduce Parameter Object** (Gaston).

Autor	Cantidad	Técnicas	Documento
Cristian	8 (3 transversales + 5)	Extract Method y Move a módulos; inyección de dependencias; Move a routers; Rename Method; Extract Function; Replace Magic Literal with Constant	`cristian_refactoring.md`
Gaston	6	**Introduce Parameter Object**; Substitute Algorithm; Replace Conditional with Lookup Table; Extract Method; Rename Variable; Remove Dead Code	`gaston_refactoring.md`
Javier	5	**Move Method**; columnas por nombre; Extract Method; Rename Variable; Rename Method	`javier_refactoring.md`
Bruno	4	**Extract Method** (conversión de filas, conteo, vencimiento, porcentaje)	`bruno_refactoring.md`

Los cambios transversales (conexión, arranque y routers) se hicieron primero y se integraron antes de que el resto empezara, para que cada uno trabajara en su propio archivo sin conflictos.

Introduce Parameter Object (Gaston)

Antes: once argumentos posicionales, en el orden del `INSERT`.

```
def guardar_solicitud(conn, estudiante_id, tipo, descripcion, prioridad,
estado,
                      responsable_id, area, fecha_alta, fecha_mod, canal, obs):
```

Después: un `dataclass` armado por nombre, y un `INSERT` con parámetros con nombre.

```
@dataclass
class NuevaSolicitud:
    estudiante_id: int
    tipo: str
    descripcion: str
    # ... resto de los campos

def guardar_solicitud(conn, solicitud):
    c = conn.cursor()
    c.execute("""INSERT INTO solicitudes (...) VALUES
(:estudiante_id, :tipo, ...)""",
              asdict(solicitud))
```

Move Method (Javier)

Antes: las transiciones permitidas, calculadas con tres `if` dentro del endpoint.

```
ok = False
if act == "PEND" and (nuevo == "ASIG" or nuevo == "CANC"):
    ok = True
if act == "ASIG" and (nuevo == "EN_PROC" or nuevo == "CANC"):
    ok = True
if act == "EN_PROC" and (nuevo == "RES" or nuevo == "RECH"):
    ok = True
```

Después: la regla vive en el dominio (`app/dominio.py`) y el endpoint la consulta.

```
TRANSICIONES = {
    "PEND": ("ASIG", "CANC"),
    "ASIG": ("EN_PROC", "CANC"),
    "EN_PROC": ("RES", "RECH"),
}

def transicion_permitida(actual, nuevo):
    return nuevo in TRANSICIONES.get(actual, ())
```

Extract Method (Bruno)

Antes: el mismo diccionario de doce campos, escrito en `buscar` y en `getSolicitud`.

```
res.append({"id": r[0], "estudiante_id": r[1], "tipo": r[2], "descripcion":
r[3],
            "prioridad": r[4], "estado": r[5], "responsable_id": r[6], ...})
```

Después: una sola función y dos usos de una línea.

```
def fila_a_solicitud(fila):
    return {"id": fila[0], "estudiante_id": fila[1], ...}

return [fila_a_solicitud(r) for r in rows]
```

Conexión inyectada (Cristian, transversal)

Antes: conexión y cursor globales, abiertos al importar el módulo.

```
conn = sqlite3.connect("solicitudes.db", check_same_thread=False)
c = conn.cursor()
```

Después: una conexión por request, inyectada; la base se prepara al arrancar la aplicación.

```
def get_db(request: Request):
    conn = conectar(request.app.state.ruta_db)
    try:
        yield conn
    finally:
        conn.close()

@router.post("/estudiantes")
def crear_estudiante(data: dict = Body(...), conn: sqlite3.Connection =
Depends(get_db)):
```

Cada documento de `docs/refactoring/` tiene el antes y después completo de todos los refactorings, el hallazgo que resuelve y el principio. Tag del cierre de la etapa: `etapa-refactoring`.

7. Pruebas y verificación

La misma suite de tests pasó sobre el código original y pasa sobre el código final, sin modificar ningún assert: esa es la evidencia de que el refactoring no alteró el comportamiento externo.

Criterio: salida idéntica. Antes de refactorizar escribimos tests que fijan lo que el sistema responde hoy, aunque sea un defecto. El comportamiento esperado queda en un comentario (`# Actual: 200. Esperado: 422.`) o en el nombre del test (`test_DEFECTO_...`). Así, un refactoring que cambia una respuesta hace fallar un test, y una corrección futura sabe exactamente qué test actualizar.

Momento	Código	Tests	Resultado
Línea de base (tag `etapa-tests`)	`main.py` original, sin cambios	64, de los cuatro integrantes	64 passed
Después de cada refactoring	cada commit de las ramas `refactor/*`	los 64 de la línea de base, más los nuevos	en verde en todos los commits
Versión final	estructura por módulos	74: los 64 sin cambios + 10 que fijan comportamiento no cubierto	74 passed

Cómo se armó la suite

- **Base aislada por test** (`tests/conftest.py`): cada test crea la aplicación sobre una base temporal nueva con los datos de prueba. Los tests no dependen del orden ni de corridas anteriores, y no tocan la base real.
- **Los errores 500** se prueban con `client_sin_excepciones`, que devuelve la respuesta en lugar de cortar el test, solo donde hace falta.
- **Cada test verifica el efecto en la base**, no solo la respuesta: en las altas, que lo guardado coincida con lo enviado; en los errores, que la cantidad de registros no cambie.
- **Tests nuevos durante el refactoring:** antes de modificar una regla sin cobertura se escribió un test que pasa sobre el código sin refactorizar (vencidas del reporte, prioridad y área por tipo). Bruno comprobó con mutaciones que esos tests fallan si la regla cambia.

Otras verificaciones

- `import main` en una carpeta vacía ya no crea ningún archivo (antes creaba la base).
- Al mover los endpoints a los routers, las líneas de código son las mismas: solo cambian de archivo.
- Gaston comparó 51 requests a `POST /solicitudes` contra la versión anterior: misma respuesta, misma fila, mismo historial y mismo log.

Evidencias: `docs/pruebas/` (resultados por integrante) y capturas de las corridas en `docs/evidencias/fase2/` y `docs/evidencias/fase3/`.

8. Code review y checklist técnico

Todo cambio entró por pull request a `dev`, con al menos una aprobación de otro integrante (`main` y `dev` están protegidas) y con merge commit para conservar cada commit como evidencia. Cada PR se revisó con el checklist de abajo.

Confirmación del code review (Javier Ovelar, coordinador): revisamos en conjunto los PRs y el checklist técnico, y confirmamos que los cambios se ejecutaron siguiendo los criterios establecidos. Se verificó que la suite pasara, que los refactorings conservaran el comportamiento original y que los cambios quedaran documentados y revisados mediante aprobación cruzada.

Checklist

#	Criterio	Aceptación
1	La suite pasa	`pytest` en verde sobre la rama combinada con `dev`
2	Refactoring seguro	Ningún assert de la línea de base modificado; si cambia una respuesta, no es refactoring
3	Un cambio por commit	Cada refactoring en su commit, con mensaje `refactor:` / `test:` / `docs:` que diga qué y por qué
4	Trazabilidad	El commit o el documento cita el hallazgo de la ficha que resuelve y el principio
5	Alcance	Solo toca la parte del autor o un módulo común acordado; sin cambios de formato ajenos al objetivo
6	Nombres y estructura	Nombres expresivos; funciones con una responsabilidad; sin código muerto ni comentado
7	Sin duplicación nueva	Usa lo que ya existe en `app/dominio.py`, `app/validaciones.py` o `app/db.py`
8	Documentación	Antes y después en `docs/refactoring/`; evidencia en `docs/evidencias/`
9	Uso de IA declarado	Fila en `docs/uso-ia/` y evidencia en `anexo/ia/`
10	Repositorio limpio	Sin `__pycache__`, bases de datos ni archivos duplicados

Observaciones y correcciones

PR	Observación	Criterio	Acción
#4 (tests de Cristian)	Los docstrings citaban mal los hallazgos (caso 6 como hallazgo 6)	4	Corregido antes del merge
#4 y #9	Dos `conftest.py` distintos: base por test y base compartida	1, 7	Se unificó en uno, con `client_sin_excepciones` para los 500
#7 (diagnóstico de Javier)	Evaluaciones inconsistentes en los casos 10, 12c y 19	4	Registrado para unificar en el informe
#10 (diagnóstico de Gaston)	Tabla de `docs/uso-ia/gaston.md` con una fila incompleta	9	Pendiente

PR	Observación	Criterio	Acción
#13 (transversales)	El commit 1 incluye el formato automático del editor sobre `main.py`	5	Aclarado en la descripción del PR
#15 (README)	El README decía que los tags iban en `dev`, pero el grupo los pone en `main`	8	Corregido
#21 (Bruno)	La declaración de IA citaba un PDF que no estaba en el repositorio	9	Reemplazado por `anexo/ia/bruno_registro_ia.md`
#19 y #22	Verificados contra `dev`: sin conflictos, suite en verde, sin asserts modificados	1, 2	Aprobados

9. Uso de inteligencia artificial

Los cuatro usamos Claude (Anthropic) en todas las etapas, y lo declaramos por integrante y por fase: herramienta, finalidad, grado de intervención y evidencia. La IA propuso, escribió o revisó; las decisiones de criterio, la ejecución de los casos, la verificación con los tests y los commits fueron nuestros.

Integrante	Etapa	Finalidad	Grado	Evidencia (anexo/ia/)
Cristian	Preparación	Resumen de la consigna, sistema base, configuración del repositorio y plan de trabajo	Alto	`cristian_claude_01_preparacion.pdf`
Cristian	Diagnóstico	Revisión del análisis inicial, guía de casos y armado de documentos	Medio	`cristian_claude_02_diagnostico.pdf`
Cristian	Pruebas	Guía para aislar la base, revisión de los tests y parte de ellos	Medio	`cristian_claude_03_pruebas.pdf`
Cristian	Refactoring	Estructura, cambios transversales y refactorings de su parte	Alto	`cristian_claude_04_refactoring.pdf`
Gaston	Coordinación	Plantillas de la ficha de smells y de la tabla de deuda	Alto	`gaston_registro_ia.md`
Gaston	Diagnóstico	Casos de prueba y borrador de la ficha	Alto	`gaston_registro_ia.md`
Gaston	Pruebas	Pasar a tests los 17 casos de la Fase 1	Alto	`gaston_registro_ia.md`
Gaston	Refactoring	Preguntas de diseño, refactorings y tests nuevos	Alto	`gaston_registro_ia.md`
Javier	Diagnóstico	Casos de prueba y armado de la ficha	Medio	`javier_claude_01_diagnostico.pdf`
Javier	Pruebas	Revisión de sus tests y verificaciones adicionales	Medio	`javier_claude_02_pruebas.pdf`
Javier	Refactoring	Pasos, código de cada refactoring y revisión	Medio	`javier_claude_03_refactoring.pdf`, `javier_claude_04_refactoring.pdf`
Bruno	Diagnóstico	Análisis del código y definición de los casos	Alto	`bruno_registro_ia.md`
Bruno	Pruebas	Escritura de sus 25 tests	Alto	`bruno_registro_ia.md`
Bruno	Refactoring	Extracciones de `consultas.py` y tests de vencidas	Alto	`bruno_registro_ia.md`

Registro de prompts: las conversaciones completas, en orden y con fecha, están en `anexo/ia/` (transcripciones en PDF o registros de pedidos, según el integrante). El detalle de qué hizo la IA y qué hizo cada uno está en `docs/uso-ia/<nombre>.md`.

Cómo la usamos: para comprender conceptos, proponer casos, revisar el trabajo y escribir código y documentación. Cada propuesta se verificó contra el sistema real (casos en Swagger UI, suite en verde) y varias se corrigieron a partir de esa verificación.

10. Conclusiones y mejoras futuras

El sistema final hace lo mismo que el inicial, pero ahora se puede cambiar, probar y revisar por partes: esa diferencia, invisible desde la interfaz, es la que el trabajo buscaba demostrar.

Conclusiones grupales

El trabajo nos mostró en concreto la diferencia entre calidad externa e interna. El sistema inicial funcionaba en los casos normales, pero tenía 47 hallazgos adentro: una conexión global que impedía aislar los tests, reglas de negocio repetidas como cadenas de `if`, columnas leídas por posición y errores que respondían con HTTP 200. Nada de eso se veía desde Swagger, y todo encarecía cualquier cambio.

Decisiones que funcionaron. Repartir el código y no las etapas hizo que los cuatro pasáramos por el diagnóstico, los tests y el refactoring de una parte propia, y que cada uno pueda defender todo el proceso. Fijar el comportamiento con 64 tests antes de tocar el código nos dio una regla objetiva: un refactoring es seguro si la suite sigue en verde sin modificar ningún assert. Resolver primero los cambios transversales (conexión, arranque y un router por área) permitió que después cada uno trabajara en su archivo sin conflictos.

Dificultades. Lo más difícil no fue el código, sino acordar criterios entre cuatro personas que diagnosticaban en paralelo: qué código HTTP corresponde a cada error, qué es prioridad alta, cómo documentar cada hallazgo. También tuvimos que unificar dos diseños distintos de `conftest.py`, y sostener la disciplina de separar refactoring de corrección: varias veces la mejora "obvia" cambiaba la respuesta del sistema y había que dejarla para después.

Límites del trabajo. Elegimos que la versión final responda exactamente igual que la inicial, para que la verificación fuera objetiva. Por eso los defectos que solo se corrigen cambiando la salida —la inyección SQL, los errores 500 y los códigos HTTP— quedaron documentados y priorizados, pero no corregidos. Es la principal deuda que dejamos, y la inyección SQL es lo primero que corregiríamos.

Relación con la materia. Los smells del catálogo aparecieron en código real y con consecuencias medibles; SRP, OCP y DIP guiaron la estructura nueva; el refactoring seguro dependió de tener tests antes de empezar; y el code review con checklist, junto con los PR obligatorios, hizo que cada cambio quedara revisado y trazable. Nos llevamos que un sistema profesional no se mide solo por lo que hace, sino por lo que cuesta cambiarlo.

Mejoras pendientes

Detalle, prioridad y tests afectados en `docs/decisiones/02_deuda_tecnica.md`.

Prioridad	Mejora
Alta	Consulta parametrizada en la búsqueda (inyección SQL)
Alta	Modelos de entrada con Pydantic: 422 en lugar de 500 y de datos inválidos aceptados
Alta	Códigos HTTP unificados (422, 404, 409, 201) en lugar de errores con 200
Alta	Restricciones `NOT NULL` y `UNIQUE` en la base; historial en las reasignaciones; reporte tolerante a fechas inválidas
Media	Filtros de búsqueda validados; servicio de notificaciones compartido

Prioridad	Mejora
Baja	Parameter Object en `buscar`; nombres pendientes (`getSolicitud`)

Conclusiones individuales

> *Cada integrante completa su párrafo (3 o 4 líneas): qué aprendió, qué le costó y qué haría distinto.*

Bruno Maza: Me tocaron la búsqueda, el detalle y el reporte, y coordinar los tests del grupo. Lo que más me llevo no es técnico: entendí cómo se trabaja de verdad sobre un mismo repositorio, con ramas, pull requests y revisiones cruzadas, que es lo más parecido al ejercicio real de la profesión que hice hasta ahora. Eso mismo fue lo que más me costó al principio, pensar la dinámica de trabajo; una vez que acordamos cómo trabajar, avanzar se volvió más llevadero. El code review fue lo que mejor funcionó: el control cruzado no se sintió como un peso sino como un acompañamiento. También vi para qué sirven los tests al aplicar Extract Method en mis tres endpoints, porque la suite en verde después de cada cambio era la señal de que no había roto nada. Usé IA de forma intensiva en todas las etapas, y lo que aprendí fue a no aceptarle nada sin verificarlo: varias cosas se las corregí, y un hallazgo del diagnóstico lo encontré yo ejecutando los casos, no en el análisis previo. Si lo hiciera de nuevo, le dedicaría más tiempo a entender cómo se construyen las pruebas, aunque fuera en un proyecto más chico. Sobre los defectos que quedaron pendientes, me parece una decisión acertada para un trabajo práctico; en una empresa, el paso siguiente sería abordarlos.

Cristian A. Morales: Me tocó la base del sistema (la conexión, las tablas, estudiantes y responsables) y los cambios que cruzaban el código de todos. Lo que más aprendí fue el valor de tener tests antes de refactorizar: armar el `conftest.py` con una base aislada por test y fijar el comportamiento actual, aunque fuera incorrecto, me dio una regla clara para saber si un cambio era seguro. También entendí en la práctica la diferencia entre refactorizar y corregir: varias mejoras que parecían obvias cambiaban la respuesta del sistema y tuvieron que quedar como deuda. Lo que más me costó fue coordinar: unificar criterios entre cuatro personas, resolver el choque de los dos `conftest.py` y aplicar los cambios transversales sin romper el trabajo de los demás (en el camino aparecieron problemas como el formateador automático del editor, que ensuciaba los diffs). Usé IA como apoyo en todas las etapas y aprendí a verificar cada propuesta con los tests antes de aceptarla. Si lo hiciera de nuevo, acordaría los criterios comunes desde el primer día y desactivaría el formato automático antes de empezar.

Javier Ovelar: Me tocó el cambio de estado y la asignación de responsable, y coordiné el code review y docs/. Ya conocía el refactoring, pero el trabajo me sirvió para reforzarlo: con Move Method saqué de estados.py la validación de transiciones, que se resolvía con tres if y una bandera, y la pasé a dominio.py como la tabla TRANSICIONES y la función transicion_permitida, junto con la constante ESTADOS_CERRADOS. Ahora los endpoints consultan la regla en lugar de decidirla: las reglas de estado quedan en un solo lugar (DRY) y para cambiar una transición alcanza con modificar la tabla, sin tocar el endpoint (OCP). También aprendí a usar SQLite en un proyecto, incluido sqlite3.Row, y a organizar el trabajo por etapas. Lo que más me costó fue la cantidad de ramas para cada instancia, que al principio me resultó caótica comparada con proyectos anteriores con menos integrantes. Usé la IA para tareas tediosas como la documentación, que después revisaba y corregía, y como asesor para verificar mi código y mis decisiones. Si lo hiciera de nuevo, armaría antes un flujograma de los endpoints para

evitar smells como esos tres if, y replantearía el uso de las ramas, aunque no sé si eso es un problema real o falta de experiencia mía.

Gastón Paschetta: Como conclusión personal, este TP me hizo valorar lo importante que es definir una buena metodología de trabajo y aprovechar las herramientas que ofrece, por ejemplo, GitHub para establecer reglas acordes a esa metodología ya acordada entre los developers del equipo. En este caso somos solo cuatro trabajando en un Trabajo Integrador, pero imagino el impacto en proyectos en producción, con equipos de 10 o más desarrolladores. Este TP nos mostró que vale la pena enfocarse en establecer criterios y un buen flujo de trabajo antes de configurar un entorno o empezar a escribir código. Siento que tener esto en cuenta nos permite ofrecer estándares de alta calidad y blindar el trabajo de un equipo ante inconvenientes o imprevistos.

11. Anexos y enlaces

Recurso	Enlace o ubicación
Repositorio	https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta
Video de defensa	https://youtu.be/AQlXwEhOipc
ZIP de anexos	https://drive.google.com/file/d/1d6STzisTCH92vtXj0yaHTJJXya87vWC_/view?usp=drive_link
Versión inicial	tag `estado-inicial`
Hitos	tags `etapa-diagnostico`, `etapa-tests`, `etapa-refactoring`, `etapa-pruebas`, `etapa-docs`
Versión final	tag `estadorefactorizado`

Carpeta	Contenido
`app/`, `main.py`	código de la versión final
`tests/`	suite de tests (`conftest.py` común y un archivo por área)
`docs/diagnostico/`	fichas de smells y resultados de la Fase 1 de cada integrante
`docs/pruebas/`	resultados de la Fase 2
`docs/refactoring/`	antes y después de cada refactoring
`docs/decisiones/`	decisiones técnicas: estructura y conexión; salida idéntica y mejoras pendientes
`docs/evidencias/`	capturas por fase e integrante
`docs/uso-ia/`	declaración de uso de IA de cada integrante
`anexo/ia/`	transcripciones y registros de las conversaciones con IA

Participación: cada integrante tiene commits propios en todas las etapas (diagnóstico, tests, refactoring y documentación), visibles en el historial y en los PRs, con su revisión por otro integrante.

Referencias

UTN. (2026). *Trabajo Práctico Integrador*. Material de Cátedra – Metodología de Sistemas II, Tecnicatura Universitaria en Programación. Universidad Tecnológica Nacional.

Anthropic. (2026). Claude (Claude Code y claude.ai, octubre y noviembre de 2026) [Large language model]. https://claude.ai